

Definitions :

Dependency Injection

Dependency Injection is a Design pattern to implement Inversion of Control(IoC). It means that responsibility of creating and managing object is transferred from application to the Spring. Spring injects dependencies in the class automatically rather than class creating dependency by itself.

For example,

GameRunner class is dependent on Game class/Game Interface. And we are injecting GameInterface to GameRunner class.

IoC/Spring Container/Spring Context :

Pojo class and configuration act as an input to Spring Container. Spring Container manages beans and its life cycle.

Spring container — Instantiates Bean (Create Java Object), Injects Dependency (via constructor, field or setter), Manages bean lifecycles (Initialization and destructions), Handles Configuration (via annotations, xml or java config).

Bean Factory - Basic IoC container.

Application Context - Advance IoC container with advance enterprise level features (i18n feature and good integration with Spring AOP).

Autowiring

Autowiring is a spring way of performing dependency injection automatically by scanning for matching beans. @Autowired.

Spring Packages

By default if you do not add anything, it will have spring-boot-starter & spring-boot-starter-test

Spring Web (spring-boot-starter-web)- To build web application which

includes Apache tomcat as default embedded server. It includes Rest as well. It replaces spring-boot-starter to spring-boot-starter-web.

Spring Boot Actuator - monitoring of application. <http://localhost/actuator>

Spring Boot Dev Tools - Live Reload for faster development time.

Spring Data JDBC - Plain JDBC using Spring Data

Spring Data JPA - JPA(like hibernate) using Spring Data

H2 Database - In memory quick database.

Validation — for validation I guess. (Bean Validation with hibernate validator)

— @NotNull @Size @Email etc comes under this.

Quick Handy Application.properties of Spring

logging.level.org.springframework= DEBUG

Actuator : management.endpoints.web.exposure.include=* or particular

one : management.endpoints.web.exposure.include=health, metrics

spring.h2.console.enabled=true

spring.datasource.url=jdbc:h2:mem:testdb

spring.jpa.show-sql = true (Shows actual query being executed)

spring.jpa.defer-datasource-initialization=true. — Schema can be created just by @Entity annotation for in memory db. No need of schema.sql as well. But if you want data.sql queries to be populated to shcema created by schema.sql talbe then enable this.

Spring automatic gzip compression for large response.

server.compression.enabled=true

server.compression.mime-types=application/json,application/xml,text/plain

server.compression.min-response-size=1024

Spring JDBC/JPA/Data JPA flow :

Spring JDBC -> Course Class (Same as table),

Repository -> with String row sql injection safe queries
queries , function with uses jdbcTemplate.update(Insert_Query,
course.getId(), course.getAuthor(),

For select queries, use RowMapper like return
jdbcTemplate.queryForObject(SELECT_QUERY, new
BeanPropertyRowMapper<>(Course.class), id);

Execute these methods mentioned in Repository class either using REST API endpoints or by command runner class.

Spring JPA -> Course Class (Same as table) mark it as @Entity. Use @Id and @Column annotations as well if required.

Create Repository with EntityManager autowired. private EntityManager entityManager; user entityManager.merge(course) for insert, entityManager.find(course.class, id), entityManager.remove(course)

Execute these methods mentioned in Repository class either using REST API endpoints or by command runner class.

Spring Data JPA ->. Course Class (Same as table) mark it as @Entity. Use @Id and @Column annotations as well if required.

No need of @Repository. Just create interface - public interface CourseSpringDataJpaRepository extends JpaRepository<Course, Integer> (where Integer is a data type of pk of table)

directly call predefined methods using restapi or by command liner... methods like

```
List<item> abcd {  
    return Repository.findAll();  
}
```

```
Optional<Item> -> aaa(@PathVariable int id). ->  
repo.findById(id);
```

save, saveAll, deleteById, findByAuthor (where author is field in my course), count etc..

Short overview about docker

=====

Docker Image (We can create with all required software and dependencies or software provider like mysql creates it.) -> Docker image gets uploaded to docker hub (public registry for all docker images) -> then anybody can download docker image and run it in container (container is a running instance like VM but lightweight and much faster).

docker pull mysql:latest — **downloads latest mysql docker image...**

docker run --name mysql-container -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=mydb -p 3306:3306 -d mysql:latest. — run that image within container and do port mapping.

You can access that container and connect with them via some port. Maps a port on the host (e.g., 8080) to a container port (e.g., 80).
Docker file is where we mention what all things which we need to create an image. How image should get created.

Spring Project Structure

=====

Main File -> @SpringBootApplication.

Model -> Where we write actual all entities corresponding to table.

Repository -> Class which interacts with DB or interface which we create which extends JpaRepository<entity, primaryKey datatype of entity>

Controller -> Where we mention all paths of api and call repository methods/ service methods if required.

[Optional] **Service** -> Where we actually write business logic to do some operation, computation.

DTO - files like ApiResponse.java. Which has all the structures for api response. Typically used with ResponseEntity to modify response before providing response to the request.

Exception - Global Exception or Custom Exceptions

Large Data Strategies

=====

Response

1. **Pagination** : create a method in repository interface which returns Page<items> findAll(Pagable pg) -> in controller method returns ResponseEntity<Page<Item>> and takes input page and size.

ResponseEntity<Page<Item>>

*getPaginatedItems(@RequestParam(defaultValue = "0") int page,
@RequestParam(defaultValue = "10") int size) {*

2. **Streaming** : Send large data chunk by chunk. Returns ResponseEntity<StreamingResponseBody>, we have to overwrite writeTo method of StreamingResponseBody interface. File data or largedata can be sent.

Request

1. Create Async method to send response and then process it in background. We have to enable asynchronous operations by @EnableAsync either over controller or for through out applicaiton over main class. @Async method needs to be created in service class and controller will call this service class method.

```
CompletableFuture<String> response =  
itemService.processItemsAsync(items);  
response.thenAccept(result -> {  
    System.out.println("Async task completed with result: " + result); //  
    Logging result or taking other actions  
});
```

2. For largefiles, take request as an input stream... HttpRequest
request; request.getInputStream()

Spring AOP
=====

Spring AOP (Aspect-Oriented Programming) is a programming paradigm that allows separating cross-cutting concerns from business logic. It helps in modularizing concerns like logging, security, transaction management, and exception handling without cluttering the core business logic.

Cross cutting concerns :

Security, Logging, Transaction Management, Performance Monitoring.

